Learning outcomes

By the end of this session, you should be able to:

- Python工作原理
- 字符串及其常用操作
- 整数和浮点数
- 布尔型
- 条件语句
- 数据容器：变量、列表和字典

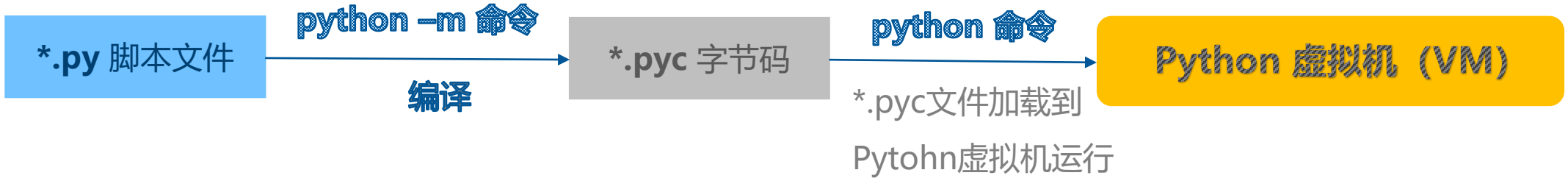
工作原理

运行方式一：脚本程序直接运行

Python 语言是典型的脚本语言，通过解析器直接运行*.py文件。所有Python脚本程序的后缀名都是以 *.py 结尾。

运行方式二：解释器编译成中间文件（字节码）后再在虚拟机中运行

- pyc 脚本程序的编译执行过程



工作原理

什么是 *.pyc 文件?

pyc是一种二进制文件，是由py文件经过解释器编译后，在磁盘上生成的文件形式，是一种byte code，py文件变成pyc文件后，加载的速度有所提高，而且pyc是一种跨平台的字节码，是由python的解析器来执行的。**pyc**的内容是跟python的版本相关的，不同版本编译后的pyc文件是不同的。

为什么需要 *.pyc 文件?

因为**py**文件是可以直接看到源码的，如果你是开发商业软件的话，不可能把源码也泄漏出去。所以就需要编译为**pyc**后，再发布出去。

交互式命令编程模式

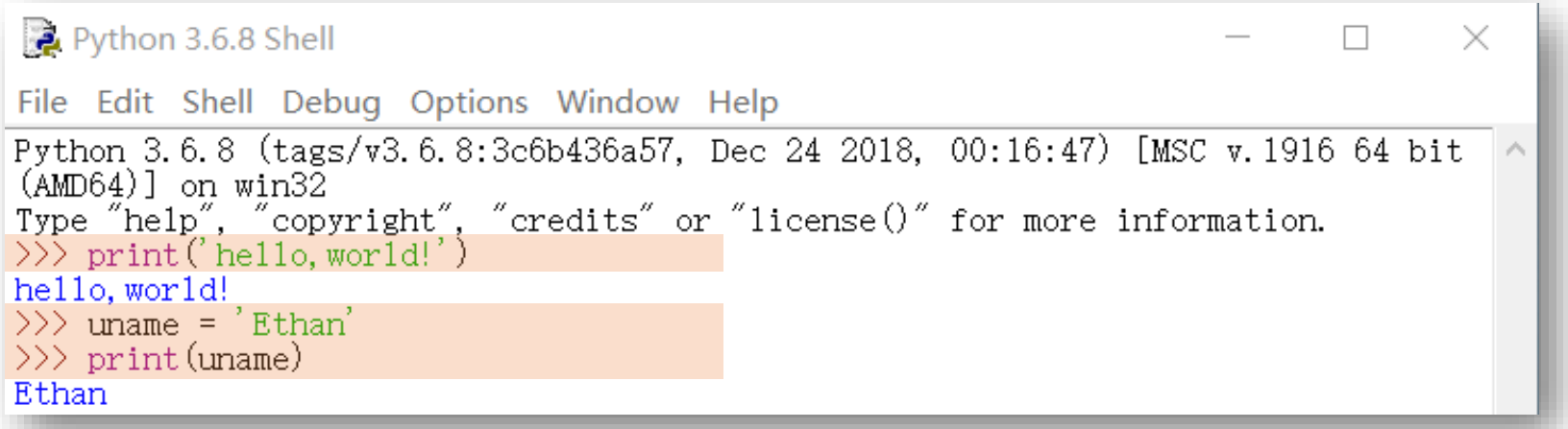
Python脚本文件是一种典型的解释型语言，其执行的方式为逐行读取逐行执行。

而交互式命令编程模式就是一种典型逐行读取执行模式。

当程序只有一行或较少的时候，这种编程模式是比较典型的应用方式。

示例代码：

右图使用Python IDLE编辑器进行编程，而该编辑器的编程模式就是典型的交互式命令编程模式。其特点就是，符号>>>就是输入交互命令的提示符，每次输入完毕后回车，该命令就被Python解析器执行。



脚本编程模式

当我们需要编写较为复杂或大段的代码的时候（特别是在使用函数编程或面向对象编程时），这样的命令式编程就显得很不舒服。

因此，Python提供了**脚本编程模式**。可以创建一个后缀名为 ***.py** 的脚本文件，将大量的代码编写到该文件中，这样便于代码的维护和更新，之后再使用交互命令执行或IDE工具运行即可。

String

字符串（概念）：

- 引号 标记地文本（单双引号都可以）
- 由0个或多个字符组成

```
[12]
1  a='data'

[13]
1  type(a)

...  str
```

String

常规操作：

- 拼接
- 复制
- 长度
- 切片
- 字符串切分
- 查看数据类型

String

拼接：

两个字符串拼在一起

```
# 拼接
a='data'
b='analysis'
a+' '+b
```

```
'data analysis'
```

String

复制：

a=‘data’

(a+‘’)*10

```
# 复制
(a+' ')*10
```

```
'data data data data data data data data data data '
```

String

长度：

- 用len函数来判断字符串长度
- a=‘data’
- len(a)

```
# 长度
len(a)
```

```
4
```

String

切片：

- 字符地索引，字符在字符串的位置，从0开始
- a=‘data’
- a[0]

```
# 切片
#字符的索引，字符在字符串的位置，从0开始
a[0]
```

```
'd'
```

String

切片:

- 左闭右开
- 若数值为负数则从右往左数
- range()函数可输出列表

- a=list(range(10))
- a
- [0,1,2,3,4,5,6,7,8,9]

Practice 1

a

[0,1,2,3,4,5,6,7,8,9]

1. 如果要取 a[2:3]，则结果为：
2. 如果要取a[-5:9]，则结果为：
3. 如果要取a[-5:-1], 则结果为：

String

字符串切分（split）：

- 将现有的字符串切分成两个单独的字符串
- c=‘data, analysis’
- c. split (‘,’)

```
#字符串切分
#split
c='data,analysis'
c.split(',')

```

```
['data', 'analysis']
```

String

查看数据类型（type）：

- c='data, analysis'
- type (c)

```
# type
# 查看数据类型
type(c)
```

```
str
```

Practice 2

1. s=‘python’; lst=[[1,2,3],[4,5,6],[7,8,9]]
   print(len(s)+len(lst))

2. list= [‘world’, ‘tourism’,’Ninghai’, ‘open’],
   则 list[2][2]的返回值是什么

Float

整数和浮点数：

- 精度
- round ()函数
- decimal模块

```
[126]  1  0.1+0.2
...    0.30000000000000004
```

```
[127]  1  0.1+0.2==0.3
...    False
```

Float

整数和浮点数：

- 在python中，浮点数在计算机硬件中表示为以 2 为基数（二进制）的小数。
- 大多数的十进制小数都不能精确地表示为二进制小数。
- 这导致在大多数情况下，你输入的十进制浮点数都只能近似地以二进制浮点数形式储存在计算机中。

```
round(0.1+0.2,1)==round(0.3,1)
```

```
True
```

Float

整数和浮点数：

使用decimal模块来提高精度

考虑精度的问题，需要使用decimal模块

```
from decimal import getcontext,Decimal
getcontext().prec=1
Decimal(0.1)+Decimal(0.2)
```

```
Decimal('0.3')
```

While循环

循环：

- **While condition**
- **statement**

```python
# While循环
# While condition :
#    statement

days=1

while days<=7:
    print("今天是周{}|".format(days))
    days+=1

```

```
今天是周1|
今天是周2|
今天是周3|
今天是周4|
今天是周5|
今天是周6|
今天是周7|
```

List

列表：

- 新建列表
- 全是数类型的列表
- 全是str类型的列表
- 混合型的列表

```python
# 新建一个空列表
list_1=[]

# 全部是数类型的列表
list_num=[1,2,3,4,5,1]

# 全部是str类型的列表
list_str=['1','2','3','4','5','1']
# 里面的不同类型的列表

list_mix=['1','2','3',1,2,3]

```

```
['1', '2', '3', 1, 2, 3]
```

List

列表：

- **列表的复制**
- **列表的合并**

```python
# 列表的复制
# list_num*2

# 列表的合并

list_num+list_str
```

✓ 0.1s

```
[1, 2, 3, 4, 5, 1, '1', '2', '3', '4', '5', '1']
```

List

列表:

- list_num=[1,2,3,4,5,1],
  list_str=['1','2','4','5','1']
- 添加数据 append (), insert ()

```
# 添加数据
# append
# insert
list_num.append(8)
```

✓ 0.4s

```
list_num.insert(2,6)
```

✓ 0.4s

```
list_num
```

✓ 0.5s

```
[1, 2, 6, 4, 5, 1, 8]
```

List

列表：

- list_num=[1,2,6,4,5,1,8],
- 删除数据：
- 根据位置删除 pop ()
- 按之删除remove()

```
list_num.pop(2)
```

✓ 0.5s

```
6
```

```
list_num.remove(8)
```

✓ 0.6s

```
list_num
```

✓ 0.3s

```
[1, 2, 4, 5, 1]
```

See you soon!

Thanks for listening!